

Neighbourhood processing: Part 1

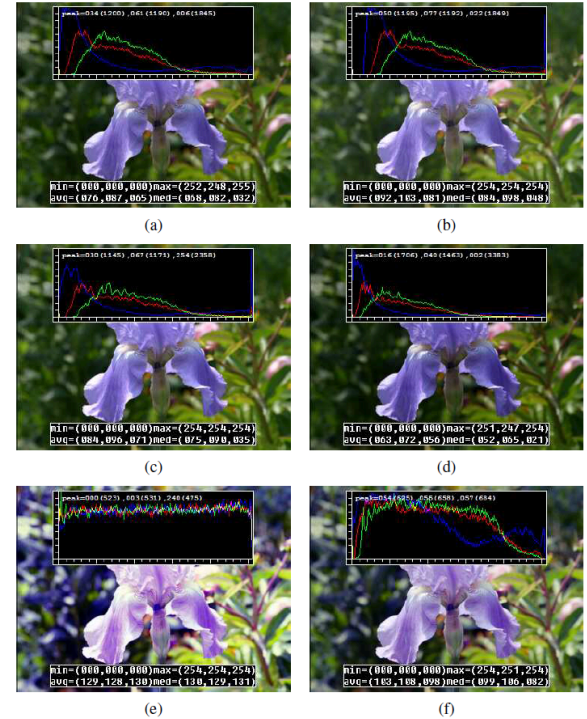
Spatial Operations

Can be subdivided into 3 broad categories:

- Single-pixel operations or point processing
- Neighborhood operations or neighborhood processing
- Geometric spatial transformations

Single-pixel operations

- Simplest possible operation on a digital image: alter the pixel intensities
- If T is a transfer function, $s = T(z)$ or
 - $z \rightarrow$ original image pixel intensity, $s \rightarrow$ mapped intensity
- Some local image processing operations:
 - original image along with its three color histograms;
 - brightness increased (additive offset, $b = 16$);
 - contrast increased (multiplicative gain, $a = 1.1$);
 - gamma (partially) linearized ($\gamma = 1.2$);
 - full histogram equalization;
 - partial histogram equalization.



Neighborhood

Neighbourhood processing means that the value of a pixel is determined using the values of the pixels around it (its neighbors) instead of using only the pixel itself. This is commonly used in filters, smoothing, edge detection, sharpening, etc.

3×3 Neighborhood

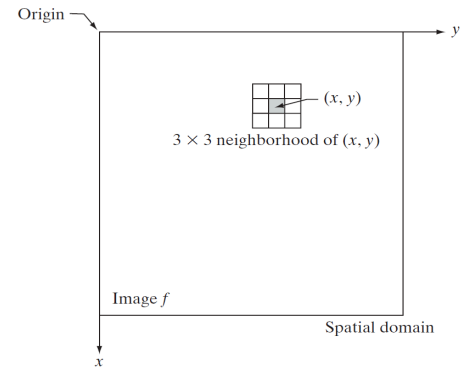
Around the center pixel, there is a **3×3 window** (small grid).

It contains **9 pixels total**:

$(x-1,y-1)$ $(x-1,y)$ $(x-1,y+1)$
 $(x,y-1)$ (x,y) $(x,y+1)$
 $(x+1,y-1)$ $(x+1,y)$ $(x+1,y+1)$

- The **middle pixel** = (x,y)
- The **8 surrounding pixels** = **neighbors**

This set of pixels is called the **3×3 neighborhood of (x, y)** .



Types of neighbors of a pixel

Assume a pixel **p** is located at (x, y) .

1. 4-Neighbors ($N_4(p)$)

These are the pixels **directly above, below, left, and right** of pixel **p**.

Coordinates:

- $(x + 1, y) \rightarrow$ below
- $(x - 1, y) \rightarrow$ above
- $(x, y + 1) \rightarrow$ right
- $(x, y - 1) \rightarrow$ left

So the **set of 4-neighbors** is:

$$N_4(p) = \{(x+1,y), (x-1,y), (x,y+1), (x,y-1)\}$$

Visual:

$(x-1,y)$
 $|$
 $(x,y-1)$ **p** $(x,y+1)$
 $|$
 $(x+1,y)$

These are called **horizontal and vertical neighbors**.

2. Diagonal Neighbors ($N_D(p)$)

These are the **four corner pixels** around p .

Coordinates:

- $(x + 1, y + 1)$
- $(x + 1, y - 1)$
- $(x - 1, y + 1)$
- $(x - 1, y - 1)$

So:

$$N_D(p) = \{(x+1, y+1), (x+1, y-1), (x-1, y+1), (x-1, y-1)\}$$

Visual:

$$(x-1, y-1) \quad (x-1, y+1)$$

$$p(x, y)$$

$$(x+1, y-1) \quad (x+1, y+1)$$

These are called **diagonal neighbors**.

3. 8-Neighbors ($N_8(p)$)

The **8-neighbors** include:

- **4-neighbors**
- **Diagonal neighbors**

So all **8 surrounding pixels**.

$$N_8(\mathbf{p}) = N_4(\mathbf{p}) + N_D(\mathbf{p})$$

Visual:

(x-1,y-1) (x-1,y) (x-1,y+1)

(x,y-1) p (x,y+1)

(x+1,y-1) (x+1,y) (x+1,y+1)

Total neighbors = **8**.

Distance Measures Between Pixels

To describe how close two pixels are, we use **distance measures**.

Let

- $\mathbf{p} = (\mathbf{x}, \mathbf{y})$
- $\mathbf{q} = (\mathbf{s}, \mathbf{t})$
- $\mathbf{z} = (\mathbf{v}, \mathbf{w})$

1. Euclidean Distance

This is the **normal geometric distance** (straight line distance).

$$D_e(\mathbf{p}, \mathbf{q}) = \sqrt{\{(x-s)^2 + (y-t)^2\}}$$

Example

$$\mathbf{p} = (2,3)$$

$$\mathbf{q} = (5,7)$$

$$D_e = \sqrt{\{(2-5)^2 + (3-7)^2\}} = \sqrt{\{(-3)^2 + (-4)^2\}} = \sqrt{\{9+16\}} = \sqrt{\{25\}} = 5$$

So the **straight-line distance = 5 pixels**.

2.D₄ Distance (City Block Distance)

Also called **Manhattan distance** (like moving along city roads).

$$D_4(p,q) = |x-s| + |y-t|$$

Example

$$p = (2,3)$$

$$q = (5,7)$$

$$D_4 = |2-5| + |3-7| = 3 + 4 = 7$$

$$\begin{array}{ccccc} & & 2 & & \\ & 2 & 1 & 2 & \\ 2 & 1 & 0 & 1 & 2 \\ & 2 & 1 & 2 & \\ & & 2 & & \end{array}$$

So **distance = 7 moves**.

4-Neighborhood

Pixels whose D_4 distance from a given pixel is 1 are called its 4-neighbors (N_4).

For pixel **(x,y)**:

$$(x,y-1)$$

$$(x-1,y) \quad (x,y) \quad (x+1,y)$$

$$(x,y+1)$$

These are called **N4 neighbors**.

3. D₈ Distance (Chessboard Distance)

Used like **king movement in chess**.

$$D_8(p,q) = \max(|x-s|, |y-t|)$$

Example

$$p = (2,3)$$

$$\begin{array}{ccccc} 2 & 2 & 2 & 2 & 2 \\ 2 & 1 & 1 & 1 & 2 \\ 2 & 1 & 0 & 1 & 2 \\ 2 & 1 & 1 & 1 & 2 \\ 2 & 2 & 2 & 2 & 2 \end{array}$$

$$q = (5, 7)$$

$$D_8 = \max(|2-5|, |3-7|) = \max(3, 4) = 4$$

So **distance = 4 moves**.

8-Neighborhood

Pixels whose D_8 distance from a given pixel is 1 are called its 8-neighbors (N_8).

$$(x-1, y-1) \quad (x, y-1) \quad (x+1, y-1)$$

$$(x-1, y) \quad (x, y) \quad (x+1, y)$$

$$(x-1, y+1) \quad (x, y+1) \quad (x+1, y+1)$$

Includes **diagonal pixels** also.

Comparison

Distance	Formula	Movement Type
Euclidean → circle	$\sqrt{\{(x-s)^2 + (y-t)^2\}}$	Straight line
D_4 → plus shape	$ x-s $	Horizontal + Vertical
D_8 → square shape	$\max(x-s , y-t)$	Horizontal + Vertical + Diagonal

Linear vs Nonlinear Operations in image processing.

Assume: $H[f(x,y)] = g(x,y)$

- $f(x,y)$ = input image
- H = operator / processing method
- $g(x,y)$ = output image

So the operator **H** transforms the input image into an output image.

Linear Operation

An operator **H** is **linear** if it satisfies **two properties**:

$$H[a_1 f_1(x, y) + a_2 f_2(x, y)] = a_1 H[f_1(x, y)] + a_2 H[f_2(x, y)]$$

Where

- $f_1(x,y), f_2(x,y)$ = two images
- a_1, a_2 = constants

This means the system follows **superposition**.

1. Additivity Property

The output of the **sum of inputs** equals the **sum of outputs**.

$$H[f_1(x,y) + f_2(x,y)] = H[f_1(x,y)] + H[f_2(x,y)]$$

Meaning

If we process two images together, the result is the same as:

1. Process each image separately
2. Add the results

Example: If

Image A $\rightarrow$ processed $\rightarrow$ Result A

Image B $\rightarrow$ processed $\rightarrow$ Result B

Then Processing (**A + B**) gives- Result = **Result A + Result B**

2. Homogeneity Property (Scaling Property)

If the input is multiplied by a constant, the output is also multiplied by the same constant.

$$H[a f(x,y)] = a H[f(x,y)]$$

Meaning

Scaling the input image scales the output the same way.

Example: If brightness of an image is doubled:

$$\text{Input} = 2f(x,y)$$

$$\text{Output: } H[2f(x,y)] = 2H[f(x,y)]$$

When Both Properties Hold

If **additivity** + **homogeneity** are satisfied $\rightarrow$ the operator is **linear**.

Examples of Linear Operations:

Common linear image processing operations:

- Averaging filter (mean filter)
- Gaussian smoothing
- Convolution
- Linear spatial filtering

Nonlinear Operations

If either property fails, the operator is nonlinear.

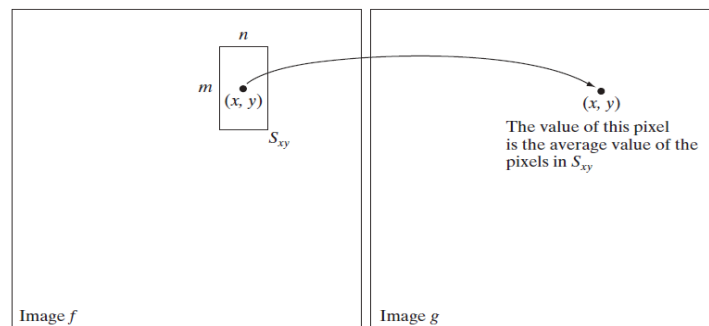
Examples of Nonlinear Operations:

Common nonlinear operations:

- Median filter
- Max / Min filter
- Histogram equalization
- Morphological operations

Example: The median filter chooses the **middle value**, not a weighted sum $\rightarrow$ so it is **nonlinear**.

Neighborhood Operations



- $S_{\{xy\}}$ is the **set of coordinates of a neighborhood** centered at an arbitrary point **(x, y)** in an image **f**.
- Neighborhood processing generates a **corresponding pixel at the same coordinates (x, y)** in the output (processed) image **g**.
- The value of this output pixel **g(x, y)** is determined by applying a **specified operation** to the pixels in the input image whose coordinates belong to **S_{xy}**.

Neighborhood Operations: Mechanism

Spatial filtering consists of:

1. A **neighborhood** (defined by the filter kernel or mask).
2. A **predefined operation** performed on the neighborhood:
 - Linear operation
 - Nonlinear operation

Filtering creates a new pixel at the same position as the center of the neighborhood, whose value is the result of the filtering operation.

A processed (filtered) image is generated as the center of the filter moves across each pixel in the input image.

Kernels (Also Known as Filters / Masks)

- A **kernel** (filter or convolution mask/matrix) is a **small matrix** used to apply effects such as **blurring, sharpening, and edge detection** to an image.
- The kernel is applied through a process called **convolution**, where it is **moved systematically across the image**, performing calculations to produce a modified image.
- Because filtering follows the same process as convolution, **filter masks are also called convolution masks**.

Filtering Process Using Kernels

- Filtering involves moving the filter/mask from point to point across the image.
- At each position (x, y), the filter response is calculated using a predefined relationship based on the mask values.
- The filter values are predefined.

Common types of filters:

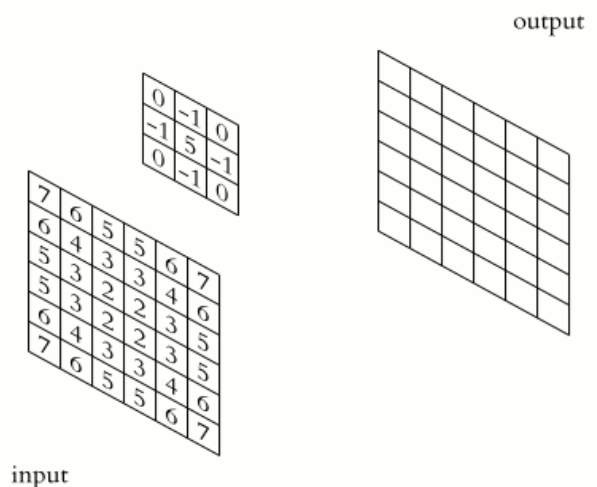
1. Smoothing filters (Low-pass filters): Reduce noise and smooth the image by removing high-frequency details.
2. Sharpening filters (High-pass filters): Enhance edges and fine details by emphasizing high-frequency components.
3. Edge detection filters: Highlight boundaries in the image where pixel intensity changes sharply.
4. Nonlinear filters: Process pixels using nonlinear operations (e.g., median) to reduce noise while preserving edges.

Key Concepts of Kernels

- Kernels are usually small matrices, such as 3×3 , 5×5 , or 7×7 . The size determines the area of the image affected by each calculation.
- The values inside the kernel determine the effect on the image. These values may be positive or negative and are used in weighted sums with pixel values.
- The kernel is placed over the image, covering part of it. Each kernel element is multiplied with the corresponding image pixel value, and the results are summed to produce the new pixel value in the output image.
- The kernel then moves to the next position (usually one pixel right or down), and the process repeats.
- This entire process is called convolution.

The animation here demonstrates the process of filtering. Here, edge Padding was applied to preserve the dimensions of the input image.

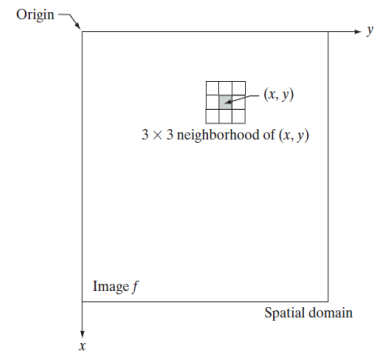
$$\text{Kernel, } w(x, y) = \begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$



Spatial Filtering

Neighborhood processing is also called **spatial filtering**.

- $g(x,y) = T[f(x,y)]$
- **Input image:** $f(x,y)$
- **Output image:** $g(x,y)$
- $T[]$ is an operator defined over a **neighborhood** of (f)



Types of operations:

- Linear operation $\rightarrow$ Linear spatial filtering
- Nonlinear operation $\rightarrow$ Nonlinear spatial filtering

The operator **T** can be applied to **a single image or multiple images**.

Example: A **3×3 neighborhood** centered at (x,y) , where the output pixel $g(x,y)$ is computed using the surrounding pixels.

Linear Spatial Filtering

- Linear spatial filtering is performed on an **$m \times n$ neighborhood** around a pixel.
- The **mask/filter/kernel size** = **$m \times n$** , where usually $(m = 2a + 1)$ and $(n = 2b + 1)$ (both odd numbers so there is a center pixel).
- The **image size** is **$(M \times N)$** .

Operation

The output pixel $g(x,y)$ is calculated using a **weighted sum of neighboring pixels**:

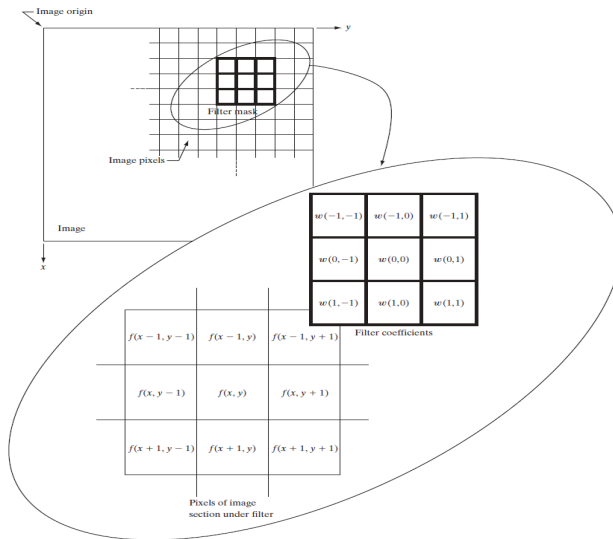
$$g(x, y) = \sum_{s=-a}^a \sum_{t=-b}^b w(s, t) f(x + s, y + t)$$

Where:

- $f(x,y)$ = input image

- $g(x,y)$ = output image
- $w(s,t)$ = filter (kernel) coefficients
- $(x+s, y+t)$ = neighboring pixels

Example: $g(x, y) = w(-1, -1) f(x-1, y-1) + w(-1, 0) f(x-1, y) + \dots + w(0, 0) f(x, y) + \dots + w(1, 1) f(x+1, y+1)$



Process shown in the figure

1. Place the filter mask over the image so its center is at (x,y) .
2. Multiply each mask coefficient (w) with the corresponding pixel value (f).
3. Add all the results.
4. The sum becomes the new pixel value $g(x,y)$.
5. Move the filter to the next pixel and repeat for the entire image.

Because the output is a linear combination (weighted sum) of neighboring pixels, it is called linear spatial filtering.

Problem: Border Pixels

When applying a **filter kernel**, pixels near the **image borders** do not have enough neighbors for the full neighborhood window.

Solution 1: Don't visit the border pixels

- The filter is applied only to pixels that have a **complete neighborhood**.
- Border pixels are **ignored**.

Result:

The **output image becomes smaller (shrunked)**.

Solution 2: Padding

Add extra pixels around the image so the filter can still be applied at the borders.

Result:

The **image size remains the same**.

Image Padding Methods

1. Zero Padding

Extend the borders by filling with **0 values**.

2. Constant Padding

Extend the borders with a **constant pixel value** (e.g., all 1s or 255).

3. Mirror Padding

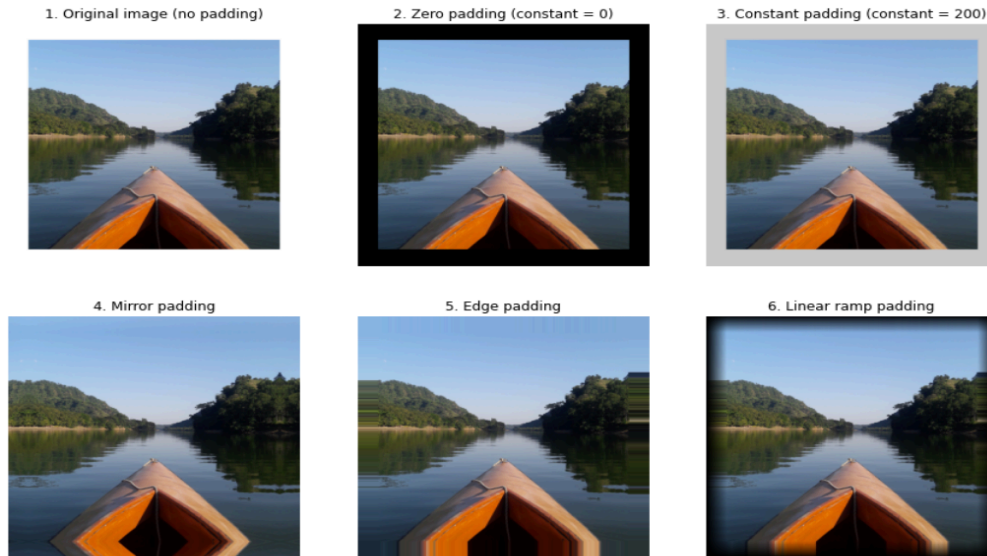
Extend the borders by **reflecting the edge pixels** of the image.

4. Clamp / Edge / Nearest Padding

Extend the borders by **repeating the nearest edge pixels**.

Image Padding (Explanation of the figure)

A **1448 × 1448 × 3** image (RGB image) is padded with **200 pixels on each side** so filtering can be applied at the borders without shrinking the image.



1. Original Image (No Padding)

- The image is used as it is.
- When filtering is applied, **border pixels lack neighbors**, which can cause **size reduction**.

2. Zero Padding (constant = 0)

- The border around the image is filled with **0 values (black pixels)**.
- Result: a **black frame** appears around the image.

3. Constant Padding (constant = 200)

- The border is filled with a **constant pixel value** (here 200).
- Produces a **uniform gray border** around the image.

4. Mirror Padding

- The border is created by **reflecting the image edges**.
- Pixels near the edge are **mirrored outward**, creating a natural continuation.

5. Edge Padding (Clamp / Nearest)

- The **edge pixels are repeated outward**.
- The border looks like a **stretched version of the edge pixels**.

6. Linear (Ramp) Padding

- The border values **gradually change (interpolate)** from the edge pixels outward.
- Produces a **smooth transition** instead of sharp borders.

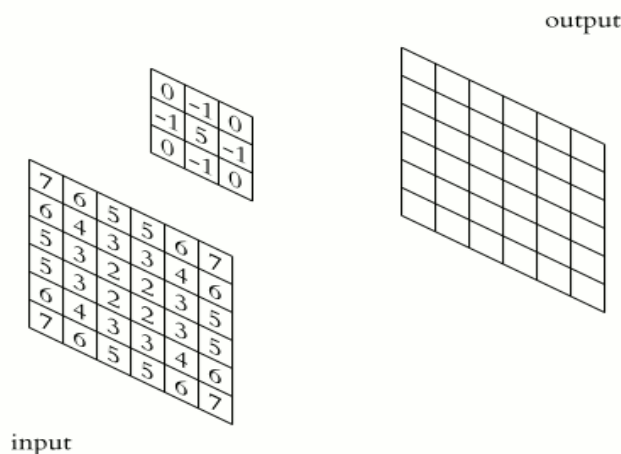
Key idea:

Padding adds extra pixels around the image so spatial filtering can process border pixels while keeping the image size unchanged.

Linear Spatial Filtering: Output shape The Variables

To calculate the output dimensions (HxW), we need five key pieces of information:

- **Input Image Shape (MxN):** The original height (M) and width (N) of the image.
- **Kernel Shape (mxn):** The dimensions of the filter. Common sizes are 3 x 3 or 5 x 5.
- **Padding (p):** The number of extra pixel layers added to the outside of the image (usually filled with zeros). This allows the filter to cover the edges of the original image.
- **Stride (s):** The "step size." A stride of 1 means the filter moves one pixel at a time; a stride of 2 means it skips every other pixel, effectively downsampling the image.



The Output Formulas

The slide provides two identical formulas for height (H) and width (W):

$$H = \text{floor}\left(\frac{M + 2p - m}{s} + 1\right)$$
$$W = \text{floor}\left(\frac{N + 2p - n}{s} + 1\right)$$

For **image padding with a kernel (filter) size**, the padding amount **p** depends on the kernel size.

$$p = (k-1)/2$$

where **k = kernel size**

Examples:

- **3 × 3 kernel** → **p = 1**
(1 pixel padding on each side)
- **5 × 5 kernel** → **p = 2**
(2 pixels padding on each side)
- **7 × 7 kernel** → **p = 3**
(3 pixels padding on each side)

Idea:

Larger kernels need **more padding** so the filter can still be centered on the border pixels while keeping the **output image size the same**.

Correlation

Correlation between two 2D signals **w(x, y)** and **f(x, y)** is defined as:

$$w(x, y) \star f(x, y) = \sum_{s=-a}^a \sum_{t=-b}^b w(s, t) f(x+s, y+t)$$

- It **measures the similarity** between two signals as one signal is **shifted (displaced)** relative to the other.
- Also called **cross-correlation**.
- It works like a **sliding dot product** (element-wise multiplication followed by summation).
- Commonly used to **search a large signal/image f(x,y)** for a **smaller known pattern w(x,y)** (feature or template).

Correlation to Convolution

- **Linear spatial filtering** is basically a **2D signal correlation operation**.
- In **correlation**, the filter $w(x,y)$ is applied directly to the image.
- If the original filter $w(x,y)$ is **rotated by 180°** and then applied to the image, the operation is called **convolution**.

So,

- **Correlation:** use the filter **as it is**
- **Convolution:** **rotate the filter 180°**, then apply it.

$$\text{Correlation: } w(x, y) \star f(x, y) = \sum_{s=-a}^a \sum_{t=-b}^b w(s, t) f(x+s, y+t)$$

$$\text{Convolution: } w(x, y) \star f(x, y) = \sum_{s=-a}^a \sum_{t=-b}^b w(s, t) f(x-s, y-t)$$

For symmetric filters, convolution and correlation both yield the same result!

Smoothing Spatial Filters

Types

- **Linear:** Box/Average filtering, Gaussian filtering
- **Non-linear:** Median filtering

Example: **3 × 3 filter** with coefficients $w_1 \dots w_9$

Linear Smoothing Filters

1. Averaging (Box) Filter

- Output = **average of all pixels** inside the kernel
- Kernel:
 $w(x,y) = 1/(m*n)$

The image:

[[0 255 255 0 0],

[0 255 255 0 0],

[0 255 255 0 0]]

Use a **3×3 averaging (box) filter**.

Kernel:

$$\frac{1}{9} \times \begin{array}{|c|c|c|} \hline 1 & 1 & 1 \\ \hline 1 & 1 & 1 \\ \hline 1 & 1 & 1 \\ \hline \end{array}$$

Take the first **3×3 neighborhood** from the image:

[[0, 255, 255],

[0, 255, 255],

[0, 255, 255]]

Multiply with the kernel (sum then divide by 9):

$$g(x,y) = (1/9)(0+255+255+0+255+255+0+255+255) \\ = 1530/9 = 170$$

Filtered pixel value = 170

So the **center pixel after averaging filter** ≈ 170 .

Benefit:

- The original pixels were **very different (0 and 255)** → very sharp change.
- Averaging gives **170**, which is a **middle value**.

So it:

- Reduces noise
- Smooths sharp intensity changes
- Creates a blur effect
- Removes high-frequency variations

In short: 170 makes the transition between black (0) and white (255) smoother.

2. Gaussian Filter

- Output = **weighted average** of pixels (center pixels have higher weight)
- Kernel:

$$w(x, y) = \exp(-(x^2+y^2)/2\sigma^2)$$

Example: **3×3 Box filter** and **3×3 Gaussian filter**

(Filters are normalized so the output pixel intensity remains unchanged)

Example: 3×3 Gaussian Filter

The image:

[[0 255 255 0 0],

[0 255 255 0 0],

[0 255 255 0 0]]

Gaussian kernel (normalized):

$$\frac{1}{16} \times$$

1	2	1
2	4	2
1	2	1

Same 3×3 image neighborhood:

[[0, 255, 255],

[0, 255, 255],

[0, 255, 255]]

Multiply and sum:

$$g(x,y) = (1/16)(0 \times 1 + 255 \times 2 + 255 \times 1 + 0 \times 2 + 255 \times 4 + 255 \times 2 + 0$$

$$= 3060/16 \approx 191$$

Filtered pixel value ≈ 191

Idea: Gaussian gives **more weight to the center pixels**, so the result is influenced more by nearby pixels than distant ones.

Advantages of Gaussian Filter

- **Better smoothing:** Reduces noise while preserving important structures.
- **Center-weighted:** Gives **more importance to the center pixel and nearby pixels**, so results are more natural than simple averaging.
- **Less blurring of edges:** Compared to the box filter, edges are **less distorted**.
- **Produces smooth transitions:** Intensities change more **gradually and naturally**.

Applications

- **Image Blurring:** smooths the image.
- **Image Sharpening:** blur is used first in some sharpening methods.
- **Image Denoising:** reduces noise.
- **Low-pass filtering:** removes high-frequency variations.

1. Gaussian filtering creates blur



- Applying a **Gaussian kernel (25×25 , $\sigma^2 = 8$)** smooths the image.
- It reduces **noise and small details**, making the image softer.

2. Kernel size controls blur strength

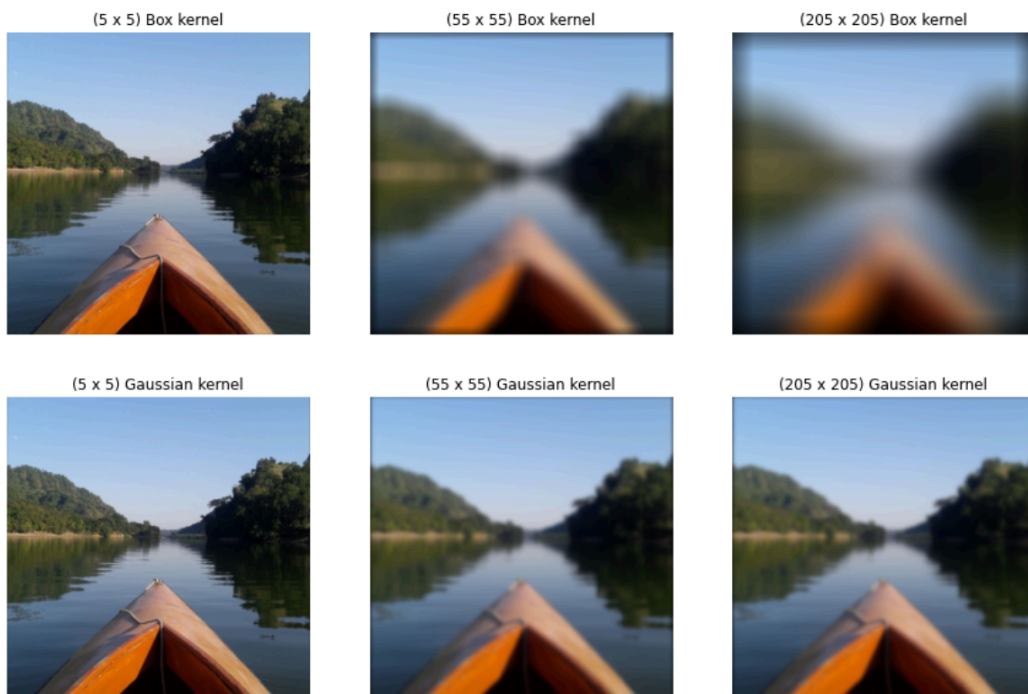


- **Small kernel (5×5)** → Slight blur.
- **Medium kernel (55×55)** → More blur.
- **Large kernel (205×205)** → Very strong blur and loss of details.

3. Larger kernel = smoother image

- As the **kernel size increases**, the image becomes **more blurred and smoother**.

4. Gaussian vs Average (Box) filtering



- **Gaussian filter preserves edges and details better.**
- **The average filter blurs everything equally**, so edges become more distorted.

Key conclusion:

Gaussian filtering is better for **noise reduction while preserving important image structures (edges)** compared to average filtering.

Order Statistic Filtering

1. Median Filtering

The output value at a pixel is the **median** of the intensity values in the neighborhood S_{xy} .

$$\hat{f}(x, y) = \text{median}_{(s,t) \in S_{xy}} \{g(s, t)\}$$

- **Best Use:** It is the gold standard for removing **salt-and-pepper noise** (impulse noise) while preserving sharp edges. Because it selects an actual pixel value from the neighborhood, it doesn't create new, "blurred" intensity values like an average filter would.

2. Max and Min Filtering

These filters pick the extreme values within the neighborhood.

- **Max Filter:** $\hat{f}(x, y) = \max_{(s,t) \in S_{xy}} \{g(s, t)\}$. This brightens the image and is useful for finding the brightest points or removing "pepper" (black) noise.

$$\hat{f}(x, y) = \max_{(s,t) \in S_{xy}} \{g(s, t)\}$$

- **Min Filter:** $\hat{f}(x, y) = \min_{(s,t) \in S_{xy}} \{g(s, t)\}$. This darkens the image and is useful for finding the darkest points or removing "salt" (white) noise.

$$\hat{f}(x, y) = \min_{(s,t) \in S_{xy}} \{g(s, t)\}$$

Example

Window:

2 5 255

3 7 6

4 8 9

- Max filter → **255**
- Min filter → **2**

3. Midpoint Filtering

This calculates the average of the maximum and minimum values in the neighborhood.

$$\hat{f}(x, y) = \frac{1}{2} \left[\max_{(s,t) \in S_{xy}} \{g(s, t)\} + \min_{(s,t) \in S_{xy}} \{g(s, t)\} \right]$$

- **Best Use:** It works best for images corrupted by **randomly distributed noise**, like Gaussian or uniform noise. It combines order statistics with a touch of averaging.

Example

Window:

2 5 8

3 7 9

4 6 10

- $z_{min}=2$
- $z_{max}=10$

Output: $(2+10)/2=6$

4. Alpha-Trimmed Mean Filtering

This is a hybrid between a mean filter and a median filter. We sort the pixels in the neighborhood, "trim" $d/2$ pixels from the lowest end and $d/2$ from the highest end, and then average the remaining pixels.

d = number of pixels we remove (trim) from the sorted neighborhood

$$\hat{f}(x, y) = \frac{1}{mn - d} \sum_{(s, t) \in S_{xy}} g_r(s, t)$$

- **Best Use:** It is highly effective when an image is plagued by **multiple types of noise** simultaneously (e.g., a mix of salt-and-pepper and Gaussian noise). By trimming the extremes, we ignore the "salt and pepper," and by averaging the rest, we smooth out the Gaussian grain.

Example:

Window values: 2, 3, 5, 6, 100, 120, 7, 8, 9

Sorted: 2, 3, 5, 6, 7, 8, 9, 100, 120

Let $d=4 \rightarrow$ remove 2 smallest & 2 largest:

Removed: 2, 3, 100, 120

Remaining: 5, 6, 7, 8, 9

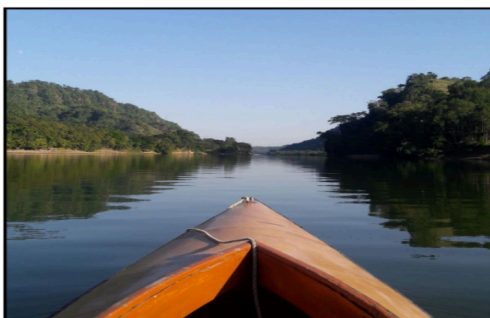
Average: $(5+6+7+8+9)/5=7(5+6+7+8+9)/5 = 7(5+6+7+8+9)/5=7$

Summary Table

Filter Type	Primary Strength	Sensitivity
Median	Salt-and-pepper noise	Excellent edge preservation
Max	Removes "pepper" noise	Brightens image
Min	Removes "salt" noise	Darkens image
Midpoint	Gaussian/Uniform noise	Vulnerable to outliers
Alpha-Trimmed	Combined noise types	Adjustable via d parameter

Important points (short):

1. Noise-corrupted image



< Original
image (left)
Noisy image >
(right)
(Gaussian
noise)



- Real images can contain **noise** during capture or transmission.
- Example shown: **Gaussian noise** (random grainy pattern).

2. Salt & Pepper Noise



< Original grayscale image (left)
Noisy image > (right) (Salt and pepper noise, pretty common for old b&w cameras)



- Appears as **random black and white pixels**.
- Common in **old cameras or transmission errors**.

3. Linear Filtering (Gaussian filter)



Gaussian filter used to denoise both noisy images (25 x 25) $\sigma^2=3$
Output is blurred, s&p not properly removed



- Used to **reduce noise by smoothing the image**.
- Kernel example: 25×25 , $\sigma^2 = 3$.
- **Problem:**
 - The image becomes **blurred**.
 - **Salt & pepper noise is not removed well.**

4. Median Filtering



Median filter is used to denoise the salt and pepper noise (can you tell the original from the filtered one?)



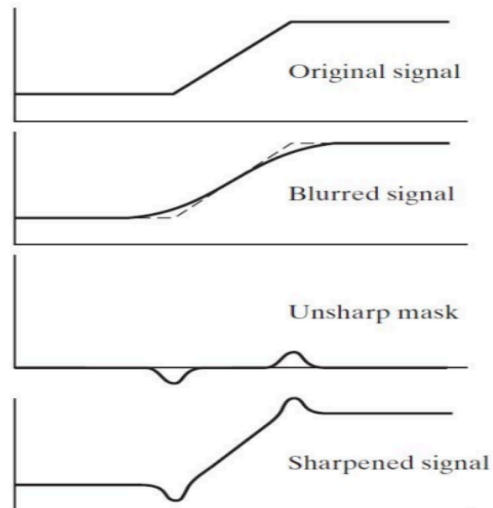
- **Non-linear filter.**
- Each pixel is replaced by the **median value of neighboring pixels.**
- **Advantages:**
 - Removes salt & pepper noise effectively.
 - Preserves edges better than linear filters.
 - Less blurring.

Key idea:

- Gaussian filter → good for Gaussian noise but causes blur.
- Median filter → best for salt & pepper noise with better edge preservation.

Unsharp Masking & High Boost Filtering

The Three-Step Process

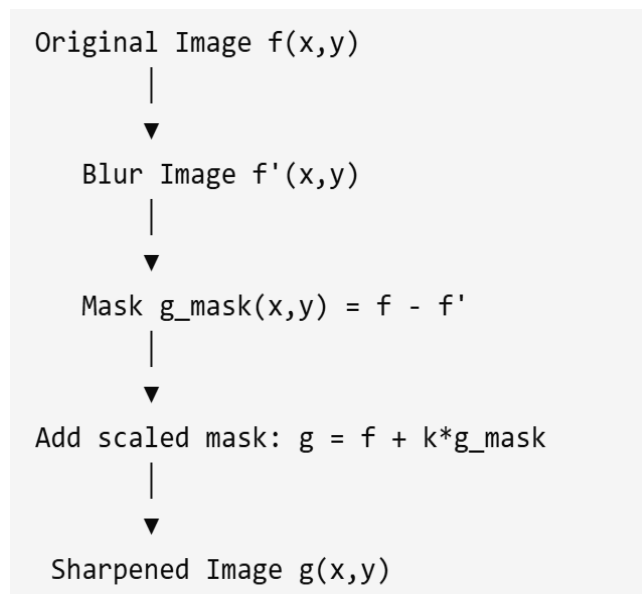


1. **Blur the Original Image:** Create a smoothed version of the image, $f'(x, y)$, typically using a Gaussian or average filter. This removes the fine details.
2. **Create the Mask:** Subtract the blurred image from the original. This "mask," $g_{\text{mask}}(x, y)$, contains only the edges and sharp transitions.
$$g_{\text{mask}}(x, y) = f(x, y) - f'(x, y)$$
3. **Combine with Original:** Add the mask back to the original image, scaled by a factor k .
$$g(x, y) = f(x, y) + k \cdot g_{\text{mask}}(x, y)$$

Unsharp Masking: $k=1$, only enhancing edges slightly.

High-Boost Filtering: $k>1$, boosts edges more strongly while keeping the original image intact.

Diagram

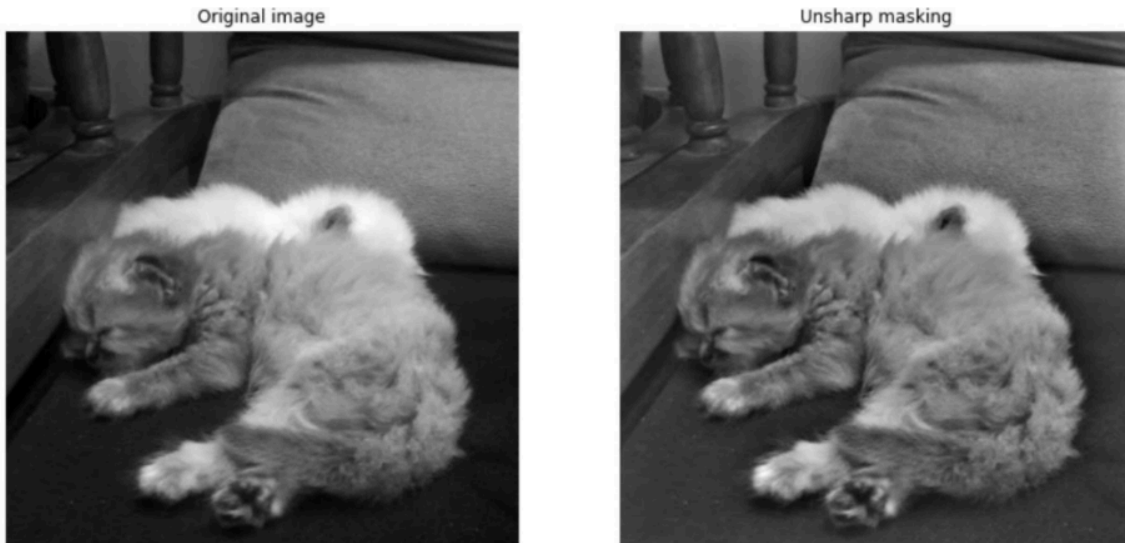


Unsharp Masking ($k = 1$)

When the weight k is exactly **1**, the process is called **Unsharp Masking**.

- **Result:** It adds the exact difference (the edges) back to the original.

- **Visual Effect:** We can see this image of the kitten. The edges of the fur and the definition of the whiskers become slightly more pronounced without changing the overall brightness of the image significantly.

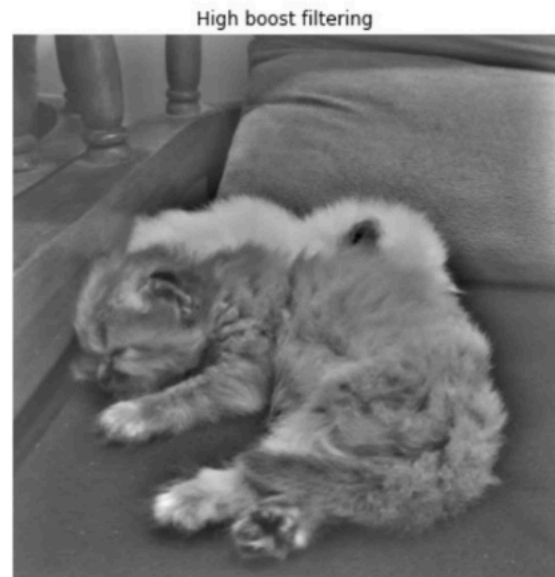


High Boost Filtering ($k > 1$)

When k is **greater than 1**, it is called **High Boost Filtering**.

- **Mechanism:** By increasing k , we are not just sharpening the edges; we are amplifying them and partially emphasizing the original image's base intensity.
- **The Equation:** Since $g_{\text{mask}} = f - f'$, the formula can be rewritten as:

$$g(x, y) = (k-1)f(x, y) + f_{\text{sharp}}(x, y)$$
 (where f_{sharp} is the unsharp masked version).
- **Visual Effect:** In the image, notice how the "High boost filtering" result looks much brighter and higher in contrast than the original. It "boosts" the high frequencies significantly, which is useful when the original image is very dull or dark.



Summary

Feature	Unsharp Masking	High Boost Filtering
Weight (k)	$k = 1$	$k > 1$
Primary Goal	Sharpen edges and fine details.	Sharpen edges while increasing overall image brightness/contrast.
Use Case	General sharpening for photography.	Enhancing dark or low-contrast images (e.g., X-rays or industrial scans).

Edge Detection

- **Edges** are **sudden changes in pixel intensity** in an image.
- Most **shape and object information** is contained in edges.
- Detecting edges makes the **image sharper and clearer**.

Common edge detection filters:

- **Prewitt**
- **Sobel**
- **Robinson Compass**
- **Kirsch Compass**
- **Laplacian**

Derivative Masks

- Edge detection works by **finding intensity differences between neighboring pixels**.
- Since an image is a **signal**, changes are calculated using **derivatives**.

Properties of derivative masks:

1. Must contain **positive and negative values** (opposite signs).
2. **Sum of all values = 0**.
3. **Higher weights** → **stronger edge detection**.

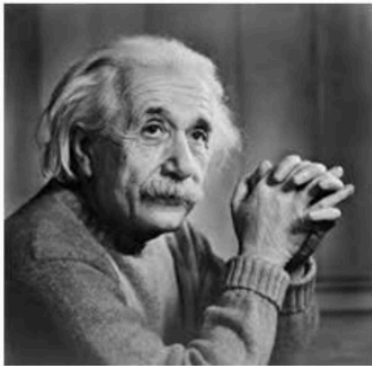
Prewitt Operator

- Uses **two masks**:
 - **Vertical mask** → detects vertical edges.
 - **Horizontal mask** → detects horizontal edges.

- The **center row/column is 0**, so it calculates the **difference between neighboring pixels**.
- Enhances edge intensity.
- Mask for detecting edges in Vertical direction:

-1	0	1
-1	0	1
-1	0	1
- Mask for detecting edges in Horizontal direction:

-1	-1	-1
0	0	0
1	1	1



Original Image



After applying Vertical Mask



After applying Horizontal Mask

Sobel Operator

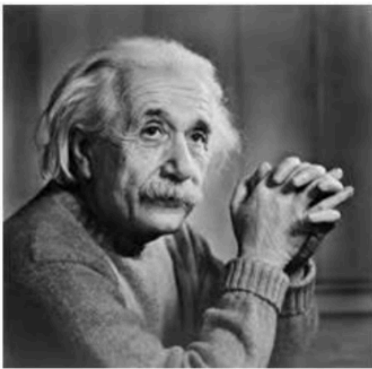
- Also uses **two masks (horizontal and vertical)**.
- Similar to Prewitt but **center pixels have weight 2**.
- Advantages:
 - **Better edge detection**
 - **Less sensitive to noise**

Example masks:

Vertical

$$\begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$

Horizontal

$$\begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$


Original Image



After applying Vertical Mask



After applying Horizontal Mask

Robinson Compass Masks

- Detect edges in **8 different directions**:
 - N, NE, E, SE, S, SW, W, NW.
- A mask is **rotated in these directions** to detect edges.

1. North (N)

$$\begin{bmatrix} 1 & 2 & 1 \\ 0 & 0 & 0 \\ -1 & -2 & -1 \end{bmatrix}$$

5. South (S)

$$\begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

2. North-East (NE)

$$\begin{bmatrix} 2 & 1 & 0 \\ 1 & 0 & -1 \\ 0 & -1 & -2 \end{bmatrix}$$

6. South-West (SW)

$$\begin{bmatrix} -2 & -1 & 0 \\ -1 & 0 & 1 \\ 0 & 1 & 2 \end{bmatrix}$$

3. East (E)

$$\begin{bmatrix} 1 & 0 & -1 \\ 2 & 0 & -2 \\ 1 & 0 & -1 \end{bmatrix}$$

7. West (W)

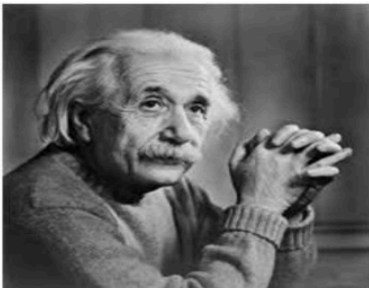
$$\begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$

4. South-East (SE)

$$\begin{bmatrix} 0 & -1 & -2 \\ 1 & 0 & -1 \\ 2 & 1 & 0 \end{bmatrix}$$

8. North-West (NW)

$$\begin{bmatrix} 0 & 1 & 2 \\ -1 & 0 & 1 \\ -2 & -1 & 0 \end{bmatrix}$$



Original Image



After applying Mask
for North West
Direction



After applying Mask
for South East
Direction

Laplacian Operator

- Uses **second-order derivatives** (others are first-order).
- Detects edges **in all directions**, not a specific one.

Types:

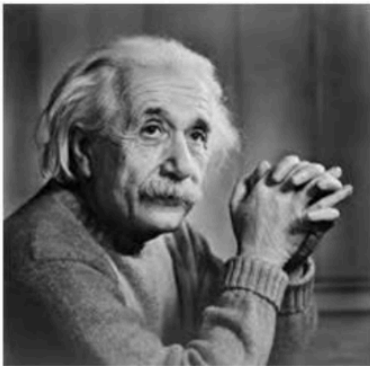
Positive Laplacian

$$\begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix}$$

Negative Laplacian

$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

- Detects **inward and outward edges**.



Original Image



After applying
Positive Laplacian
Operator



After applying
Negative Laplacian
Operator

Key idea:

- Prewitt & Sobel → detect horizontal/vertical edges

- Robinson → detects edges in 8 directions
- Laplacian → detects edges in all directions using second derivative

Geometric Transformations

1. Definition

- **Geometric transformations** change the **spatial position of pixels** in an image.
- They modify **where pixels are located**, not their intensity values.

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \mathbf{A} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} = \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

2. Rubber-sheet analogy

- Imagine the image printed on a **rubber sheet**.
- The sheet can be **stretched, shrunk, rotated, or shifted**, changing the pixel arrangement.

3. Coordinate mapping

- (x, y) → coordinates in the **original image**
- (x', y') → coordinates in the **transformed image**

4. Affine Transformations

Common geometric transformations include:

- **Scaling** → change image size

- **Translation** → move image position
- **Rotation** → rotate the image
- **Shearing** → slant the image

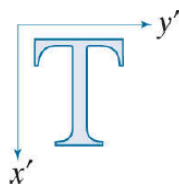
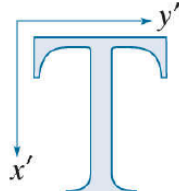
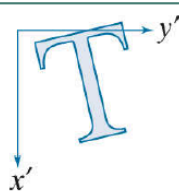
5. Key property of affine transformation

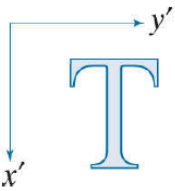
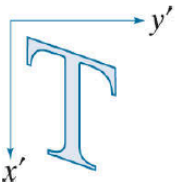
- **Preserves straight lines and points**
- Parallel lines remain **parallel** after transformation.

Key idea:

Geometric transformations **rearrange pixel positions to change image shape, size, or orientation** while maintaining basic geometric structure.

Geometric Transformations

Transformation Name	Affine Matrix, A	Coordinate Equations	Example
Identity	$\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$	$x' = x$ $y' = y$	
Scaling/Reflection (For reflection, set one scaling factor to -1 and the other to 0)	$\begin{bmatrix} c_x & 0 & 0 \\ 0 & c_y & 0 \\ 0 & 0 & 1 \end{bmatrix}$	$x' = c_x x$ $y' = c_y y$	
Rotation (about the origin)	$\begin{bmatrix} \cos\theta & -\sin\theta & 0 \\ \sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{bmatrix}$	$x' = x \cos\theta - y \sin\theta$ $y' = x \sin\theta + y \cos\theta$	

Transformation Name	Affine Matrix, A	Coordinate Equations	Example
Translation	$\begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix}$	$x' = x + t_x$ $y' = y + t_y$	
Shear (vertical)	$\begin{bmatrix} 1 & s_v & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$	$x' = x + s_v y$ $y' = y$	
Shear (horizontal)	$\begin{bmatrix} 1 & 0 & 0 \\ s_h & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$	$x' = x$ $y' = s_h x + y$	